

(10 EC pts) 1. **L^AT_EX** for dummies (optional extra credit).

- (a) The following snippet is about some key features of the big-O notation extracted from [Handout 1](#).

For (non-negative) functions $f(n)$ and $g(n)$, we say " $f(n) = O(g(n))$ " if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. The key features here are:

- "there exists a positive constant $c \dots f(n) \leq c \cdot g(n)$ ": this says that $f(n)$ is *upper bounded* by some *constant multiple* of $g(n)$. That is, O -notation "hides," or ignores constant factors. Note also that it captures only an *upper bound*: $f(n)$ might actually be *much smaller* than $c \cdot g(n)$, or not. A big-O bound by itself says nothing about which is the case.
- "there exists a positive constant $n_0 \dots$ for all $n \geq n_0$ ": this says that the upper bound holds for all n *above some constant "threshold."* However, it says nothing about the relationship between $f(n)$ and $g(n)$ *below* the threshold; it could be that $f(n)$ greatly exceeds (even a huge multiple of) $g(n)$ in that range. Moreover, the threshold n_0 can be *any constant*—it could be one, or ten, or a billion, or a googolplex—and the notation hides its exact value.

- (b) The following snippet is about [Fibonacci numbers](#).

The Fibonacci numbers may be defined by the recurrence relation

$$F_n = F_{n-1} + F_{n-2}$$

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

Input: a natural number n
Output: the n -th Fibonacci

```

1: function FIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   if  $n = 1$  then
5:     return 1
6:   return FIB( $n - 1$ ) + FIB( $n - 2$ )

```

2. Read the syllabus!

- (5 pts) (a) Suppose you attend 9 out of 14 discussion sessions and submit both course evaluation receipts. Fill in the table below to calculate the weights of each component of your final grade.

Component	Weight (%)
Homework	40%
Exams	59%
Discussion Attendance	0%
Course Evaluations	1%
Total	100

Based on your calculation, explain how attending 9 discussion impacts the weight distribution of your final grade compared to attending 10 or more discussions.

- (5 pts) (b) Below is a table showing your scores for 12 homework assignments. Assume one homework was not submitted and one homework was submitted late.

Homework #	Raw Score (%)	Notes
1	100	
2	80	
3	90	
4	0	Not submitted
5	95	
6	70	
7	85	
8	88	
9	92	
10	50	
11	94	
12	90	Submitted late

Calculate your average homework grade after applying the drop policy (dropping the two lowest scores) and the 5% late penalty.

Solution: 87.9%

- (5 pts) (c) Name three different ways you can ask a question about course content in this class. How are you more likely to ask questions?

Solution:

- discussions
- office hours

- piazza

I will likely ask in discussions

3. Welcome to EECS 376!

If you haven't already, please read [Handout 1](#) (Running Times and Asymptotics) and apply it to the next three problems.

To express our excitement for the start of EECS 376, we present an algorithm that prints "Welcome to EECS 376!" multiple times, based on two input parameters n and k . You may assume that the PRINT operation runs in $O(1)$.

Input: positive integers n and $k \in \{1, 2, \dots, n\}$, both represented in base-2 (binary)

```
1: function WELCOME( $n, k$ )
2:   for  $i = 1, 2, \dots, k$  do
3:     for  $j = 1, 2, \dots, n - k$  do
4:       PRINT("Welcome to EECS 376!")
```

- (3 pts) (a) Identify the value of k that induces the *most* "Welcome to EECS 376!" printed by the algorithm, in terms of n .

Solution: There are $k(n - k)$ iterations since this is basically $\sum_1^k \sum_1^{n-k} 1 = k(n - k)$

Thus, we get that $f(k) = kn - k^2$

The formula for the vertex of a quadratic is $x = \frac{-b}{2a}$ where a polynomial is $ax^2 + bx + c$

Thus, the max number of iterations is when $k = \frac{n}{2}$ and since a quadratic has symmetry across the point of its vertex if n is odd you can either round up or down.

Thus, the max number of iterations is when $k = \lceil n/2 \rceil$

- (4 pts) (b) Based on your answer in [Part a](#), give the *tightest correct asymptotic* (big- O) bound, as a function of n , on the worst-case number of "Welcome to EECS 376!" printed by the algorithm.

Solution: The number of prints we talked about in (a) is the function $f(k) = kn - k^2$ and from part (a) we know the worst number of iterations is when $k = \lceil n/2 \rceil$ thus $f(\lceil n/2 \rceil) \leq (n/2)(n/2) = n^2/4 \leq n^2$ Thus, the total number of iterations is bounded above by $O(n^2)$

As for showing this is the tightest bound we can also show it is $\Omega(n^2)$

If n is even then $f(\lceil n/2 \rceil) = \frac{n^2}{4}$

If n is odd then $f(\lceil n/2 \rceil) = f((n+1)/2) = \frac{n^2 - 1}{4}$

Thus in both cases $f(\lceil n/2 \rceil) \geq n^2/4$ thus it is also $\Omega(n^2)$

Thus the number of iterations is $\Theta(n^2)$

- (4 pts) (c) Determine whether the algorithm is *efficient*, i.e., runs in at most polynomial time with respect to the input size. Briefly explain your answer.

Reminder: The input size of an integer is defined as the number of bits needed to represent it in binary.

Solution: Let S be the size of the integer n . Thus, we know that $S = \lfloor \log_2 n \rfloor + 1$ for $n > 0$ and 1 for $n = 0$

Thus for large enough n we know that

$$S - 1 \leq \log_2 n < S$$

$$2^{S-1} = (1/2)2^S \leq n < 2^S$$

Thus $n = \Theta(2^S)$, thus in terms of size we know from (b) the number of iterations is $\Theta(n^2)$ thus the size is $\Theta(4^S)$

- (4 pts) (d) Does your answer in [Part c](#) change if n and k are represented in base-10 (decimals) instead? Briefly justify your answer.

Hint: You may want to use the change-of-base formula for logarithms for any positive real numbers a , b , and c , where $b \neq 1$ as well as $c \neq 1$:

$$\log_b a = \frac{\log_c a}{\log_c b}$$

Solution: This is the same proof from (c) but let $S = \lfloor \log n \rfloor + 1$ thus you get that $n = \Theta(10^S)$ and thus the size is $\Theta(10^{2S})$

4. **Lumpy array.** In lecture, we said that you can measure the “size” of an array as the number of elements in it, but we also said the “size” of an input is the number of bits needed to represent it. Consider the following example algorithm on an array:

Input: array A of n integers
Output: sum of all elements in the array

```

1: function SUM( $A$ )
2:    $S \leftarrow 0$ 
3:   for  $i = 1, 2, \dots, n$  do
4:      $S \leftarrow S + A[i]$ 
5:   return  $S$ 

```

- (5 pts) (a) For this part, we will assume that each element of the array is the same fixed size, and the “size” of our input will be considered the number of elements in the array, n . Find the big-O runtime of this algorithm (in terms of n) and state whether the function SUM is efficient, i.e., runs in polynomial time with respect to the input size (n).

Note: Because all elements of the array are the same size, we will treat addition as runnings in constant time, $O(1)$.

Solution: Lines 2,5 are both $O(1)$ operations, so we can ignore them in our analysis. Thus for the number of iterations let f be the function. We get that $f(n) = n \leq n$ thus $f(n) = O(n)$, and since what it is doing in the loop is a $O(1)$ operation we can ignore it thus the algorithm runs in $O(n)$ time. Thus, this function is efficient since $f(n) = O(n^c)$ for $c \in \mathbb{N}$

- (5 pts) (b) Suppose instead, that we measure the “size” of our array as the number of bits used to represent it, call it b . Here, the elements of the array may be of different (arbitrarily large) sizes, so we will treat addition as running in linear time (to sum numbers with k bits, it takes $O(k)$ time; you could think of this as taking finite time to add/carry each place value). Determine whether in this interpretation, our algorithm is still efficient, running in polynomial time with respect to the input size (now b).

Hint: If our input has b bits, find an upper bound on the number of elements that could be in the array and the number of bits in any one element of the array. Note that your big-O bound for the runtime of the algorithm does not need to be a tight bound.

Solution: Let b_i be the number of bits to represent $A[i]$ thus we have that

$$b = \sum_{i=1}^n b_i$$

In the min case we have that each b_i uses 1 bit. Thus, $n \leq b$.

Thus, since the loop runs $n \leq b$ times we get that the number of iterations is $O(b)$

Then it takes $O(b)$ work for each addition since $b_i \leq b$ for all $i \in \{1, \dots, n\}$

Thus, we get that the total runtime is $O(b^2)$ since we do $n \leq b$ iterations with $b_i \leq b$ work.

5. Prof. Upsilon and their oopsie claims.

Prof. Υ is a brilliant but occasionally absent-minded computer scientist, known for their groundbreaking insights into algorithms—and the occasional “oopsie” moments. Recently, Υ came up with three algorithms with the following running times:

- Algorithm X has running time $T_X(n) = O(n^2)$
- Algorithm Y has running time $T_Y(n) = \Theta(n \log n)$
- Algorithm Z has running time $T_Z(n) = \Theta(n)$

As usual, all running times are stated in terms of the *worst case* for inputs of size n . That is, $T_X(n)$ is the *maximum* number of steps for which X runs, taken over all inputs of size n (and similarly for T_Y, T_Z).

Now, Prof. Υ has made a series of claims about these algorithms. State, with proof, whether Υ 's claim is necessarily true, necessarily false, or not necessarily either based on the given information. If it is not necessarily either, give an example when it is true and an example when it is false.

- (6 pts) (a) “We can upper bound the runtime of Y by $O(n^2)$.”

Solution: Since $T_Y(n)$ is $\Theta(n \log n)$ we get that there exists constants c_1, c_2 , and n_1 s.t.:

$$c_1 n \log n \leq T_Y(n) \leq c_2 n \log n \quad \text{for all } n \geq n_1$$

Since $\lim_{n \rightarrow \infty} \frac{\log(n)}{n} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$ thus this limit is finite thus $\log(n) \in O(n)$ thus there exists constants c_3, n_2 s.t. $\log(n) < c_3 n$ for all $n > n_2$

$$c_1 n \log n \leq T_Y(n) \leq c_2 n \log n \leq c_2 c_3 n^2 \quad \text{for all } n \geq \max(n_1, n_2)$$

Thus we have that $T_Y(n) \in O(n^2)$

- (8 pts) (b) “On every input, Y runs faster than X .”

Solution: This is both true and false, depending on the functions $T_X(n)$ and $T_Y(n)$. Since $T_X(n) \in O(n^2)$, this means $T_X(n) \leq c_1 n^2$ for some constant $c_1 > 0$ and all $n > n_0$. For example, $T_X(n)$ could be 1, since $1 \leq n^2$ for all $n > 1$, thus $T_X(n) \in O(n^2)$. Similarly, suppose $T_Y(n)$ is in $\Theta(n \log n)$, which means $c_2 n \log n \leq T_Y(n) \leq c_3 n \log n$ for some constants $c_2, c_3 > 0$ and all $n > n_0$. For example, $T_Y(n)$ could be $n \log n$. Thus, $c_2, c_3 = 1$, and $n_0 = 0$

Now consider a specific input $n = 10$:

$$T_X(10) = 1, \quad T_Y(10) = 10 \log 10 = 10.$$

In this case, $T_Y(n)$ runs slower than $T_X(n)$.

However, suppose $T_X(n) = n^2 + 1$, which satisfies $T_X(n) \leq 2n^2$ for $n > 1$. Similarly to above, let $T_Y(n) = n \log n + 1$.

Define $g(n) = n^2 - n \log n$, and compute its derivative:

$$g'(n) = 2n - \frac{\ln(n) + 1}{\ln(10)}.$$

To show that $g'(n) > 0$, consider the function $h(x) = e^{2\ln(10)x-1} - x$. Compute its derivatives:

$$h'(x) = 2\ln(10)e^{2\ln(10)x-1} - 1, \quad h''(x) = 4(\ln(10))^2 e^{2\ln(10)x-1}.$$

Since $e^y > 0$ for all $y \in \mathbb{R}$, we know $h''(x) > 0$ for all $x \in \mathbb{R}$. Thus, $h'(x)$ is increasing for all $x > 0$. Also, $h'(0) > 0$, so $h'(x) > 0$ for all $x > 0$, implying $h(x) > 0$ for all $x > 0$.

Therefore:

$$e^{2\ln(10)x-1} > x \quad \text{for all } x > 0.$$

This implies:

$$2\ln(10)x - 1 > \ln(x), \quad \text{or equivalently, } 2x > \frac{\ln(n) + 1}{\ln(10)} \quad \text{for all } x > 0.$$

Thus, $g'(n) > 0$ for all $n > 0$, and since $g(0) > 0$, we conclude $g(n) > 0$ for all $n > 0$. Therefore:

$$n^2 + 1 > n \log n + 1 \quad \text{for all } n > 0.$$

Hence, $T_X(n)$ runs slower than $T_Y(n)$ on every input $n > 0$.

(10 pts) (c) “For all large enough n , there is an input of size n for which Y runs faster than X .”

Solution: This is both true and false.

Since $T_Y(n) \in \Theta(n \log n)$.

By definition, there exist constants $c_1, c_2 > 0$ and $n_0 > 0$ such that:

$$c_1 n \log n \leq T_Y(n) \leq c_2 n \log n \quad \text{for all } n \geq n_0.$$

Since $T_X(n) \in O(n^2)$.

This means there exists a constant $c_3 > 0$ and $n_1 > 0$ such that:

$$T_X(n) \leq c_3 n^2 \quad \text{for all } n \geq n_1.$$

Let $T_X(n) = 1$ for all n . Clearly, $T_X(n) \in O(n^2)$, because:

$$T_X(n) = 1 \leq c_3 n^2 \quad \text{for } c_3 = 1 \text{ and all } n > 1.$$

For $T_X(n) = 1$, we know $T_Y(n) \geq c_1 n \log n$ for all sufficiently large n :

$$T_Y(n) > T_X(n) \quad \text{for all sufficiently large } n,$$

because $c_1 n \log n \rightarrow \infty$ as $n \rightarrow \infty$, while $T_X(n) = 1$ is constant. Thus, this is false in this example.

Use the second case in part (b) to show this is true.

- (10 pts) (d) “For all large enough n , there is an input of size n for which Z runs faster than Y .”

Solution: Since $Z(n) \in \Theta(n)$, there exist constants $c_1, c_2 > 0$ and $n_0 > 0$ such that:

$$c_1 n \leq Z(n) \leq c_2 n, \quad \text{for all } n \geq n_0.$$

Since $Y(n) \in \Theta(n \log n)$, there exist constants $c_3, c_4 > 0$ and $n_1 > 0$ such that:

$$c_3 n \log n \leq Y(n) \leq c_4 n \log n, \quad \text{for all } n \geq n_1.$$

Using the bounds for $Z(n)$ and $Y(n)$, we find:

$$\frac{c_1 n}{c_4 n \log n} \leq \frac{Z(n)}{Y(n)} \leq \frac{c_2 n}{c_3 n \log n}, \quad \text{for all } n > \max(n_0, n_1)$$

Simplifying the bounds:

$$\frac{c_1}{c_4 \log n} \leq \frac{Z(n)}{Y(n)} \leq \frac{c_2}{c_3 \log n}, \quad \text{for all } n > \max(n_0, n_1)$$

As $n \rightarrow \infty$, $\log n \rightarrow \infty$, so both the lower and upper bounds approach 0:

$$\frac{c_1}{c_4 \log n} \rightarrow 0 \quad \text{and} \quad \frac{c_2}{c_3 \log n} \rightarrow 0.$$

By the Squeeze Theorem:

$$\lim_{n \rightarrow \infty} \frac{Z(n)}{Y(n)} = 0.$$

Since $\lim_{n \rightarrow \infty} \frac{Z(n)}{Y(n)} = 0$, we conclude that $Z(n) \in O(Y(n))$. Thus, for all large enough n , there is an input of size n for which Z runs faster than Y .

6. Set of proofs by contra-

Recall that a *set* is a collection of distinct elements. Below are some important set notations:

- **Elements of a set:** $x \in A$ means x is an element of set A .
- **Complement of a set:** $\overline{A}$ ¹ refers to the set of all elements in the universal set U that are not in A .
- **Subset:** $A \subseteq B$ means every element of A is also an element of B .
- **Intersection of sets:** $A \cap B$ is the set of elements that are in both A and B .
- **Union of sets:** $A \cup B$ is the set of elements that are in A , in B , or in both.
- **Set difference:** $A \setminus B$ ² is the set of elements that are in A but not in B . We can also write it as $A \cap \overline{B}$.

(5 pts) (a) Recall the transitive property of the subset relation:

If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.

Prove the above property by contradiction.

Solution: Assume $A \subseteq B$ and $B \subseteq C$
 Assume for contradiction $A \not\subseteq C$
 Thus let $x \in A$ thus $x \in B$ since $A \subseteq B$
 Thus $x \in C$ since $B \subseteq C$; however, since $A \not\subseteq C$ this implies $x \notin C$ thus this is a contradiction thus $A \subseteq C$

(5 pts) (b) Recall the DeMorgan's Law, which says:

If $x \in \overline{A \cap B}$, then $x \in \overline{A} \cup \overline{B}$.

State the contrapositive of the statement. Then, prove the contrapositive.

Solution: If $x \notin \overline{A} \cup \overline{B}$, then $x \notin \overline{A \cap B}$
 Assume $x \notin \overline{A} \cup \overline{B}$ thus $x \notin \overline{A}$ and $x \notin \overline{B}$
 Since if $x \in \overline{A}$ then it would be in $\overline{A} \cup \overline{B}$ and vice versa.
 Thus, since $x \notin \overline{A}$ this means it is in A and same for B thus $x \in A \cap B \implies$ it is not in $\overline{A \cap B}$

¹In some texts, you may encounter notations like A^C , A^c , A' , etc. to refer to the complement of A .

²Some texts may use $A - B$ to denote set difference.

7. Euclid's algorithm, extended.

Given two integers x, y (that are not both zero), one may wonder what values can be obtained by summing integer multiples of x and y , i.e., expressed as $ax + by$ for some (not necessarily positive) integers a, b . It is not too hard to see that it is impossible to obtain any positive integer *smaller* than their GCD $g = \gcd(x, y)$, because $ax + by$ must be divisible by g . A less obvious fact is that the GCD *can* be obtained in this way; this theorem is known as *Bézout's identity* (pronounced "BAY-zoo").

In this problem, you will modify the standard Euclidean algorithm to output not only $g = \gcd(x, y)$ itself, but also a pair (a, b) of integers for which $ax + by = g$. (As we will see later, this is a very important tool for cryptography.) Such integers are called *Bézout coefficients* for x, y . We give most of the algorithm below:

Input: integers $x \geq y \geq 0$, not both zero

Output: a triple (g, a, b) of integers where $g = \gcd(x, y) = ax + by$

```
1: function EXTENDED_EUCLID( $x, y$ )
2:   if  $y = 0$  then
3:     return  $(x, 1, 0)$  // Base case:  $1x + 0y = x = \gcd(x, 0)$ 
4:   else
5:     Divide  $x$  by  $y$ , writing  $x = qy + r$  for integer quotient  $q$  and remainder  $0 \leq r < y$ 
6:      $(g, a', b') \leftarrow \text{EXTENDED\_EUCLID}(y, r)$ 
7:     [FOR YOU TO DETERMINE: compute appropriate  $a, b$ ]
8:     return  $(g, a, b)$ 
```

(8 pts)

- (a) State what a and b should be on Line 7, and prove that the output is correct, i.e., that (1) $g = \gcd(x, y)$, and (2) $ax + by = g$.

Hint: by recursion/induction, we know that $g = \gcd(y, r)$ and $a'y + b'r = g$.

Solution: $x = qy + r \implies r = x - qy$

$g = a'y + b'r \implies g = a'y + b'(x - qy) \implies g = (a' - b'q)y + b'x$.

Thus, we let $a = b'$ and $b = a' - b'q$.

For (1), since g is the $\gcd(y, r)$, we know that $g \mid y$ and $g \mid r$, so:

$y = gk$ and $r = gc$, for $k, c \in \mathbb{Z}$.

Substituting into $x = qy + r$, we have:

$x = g(qk + c)$.

Thus, $g \mid x$. Let $g' = \gcd(x, y)$. Since $g \mid x$ and $g \mid y$, we conclude:

$g \geq g'$.

On the other hand, since $r = x - qy$ and $g' \mid x$ and $g' \mid y$, it follows that $g' \mid r$. Hence:

$g' \leq g$.

Since $g \geq g'$ and $g \leq g'$, we conclude $g = g'$. Thus: $\gcd(x, y) = \gcd(y, r)$.

For (2), reversing the argument starting from $g = (a' - b'q)y + b'x$ yields the desired result.

(8 pts)

- (b) Run the Extended Euclid algorithm by hand to find Bézout coefficients for the input $(x, y) = (376, 281)$, and show that the output is correct. (You may use a calculator/computer

Homework 1

only for the division steps.) Fill in the table below with a ‘trace’ of the execution, i.e., all the variables’ values in all the iterations. Also include the potential values $s = x + y$, the ratios (as fractions) of potentials s_{i+1}/s_i for adjacent iterations, and Y/N indications of whether $s_{i+1}/s_i \leq 2/3$.

i	input		division		rec ans		output		$s \quad s_{i+1}/s_i \quad \leq 2/3 ?$		
	x	y	q	r	a'	b'	a	b			
0	376	281									

The entries labeled ‘input’, ‘division’, s , and s_{i+1}/s_i should be filled from top to bottom, corresponding to the recursive calls; the ‘recursive answer’ and ‘output’ entries will need to be filled from bottom to top, corresponding to the ‘post-processing’ (Line 7) of each recursive call’s results. The entries for a, b should match those of a', b' one row above. Put ‘-’ for any entries that are not defined due to the base case.

Solution:

i	input		division		rec ans		output		$s \quad s_{i+1}/s_i \quad \leq 2/3 ?$		
	x	y	q	r	a'	b'	a	b			
0	376	281	1	95	-24	70	70	94	657	376/657	T
1	281	95	2	91	22	-24	-24	70	376	186/376	T
2	95	91	1	4	-1	22	22	-24	186	95/186	T
3	91	4	22	3	1	-1	-1	22	95	7/95	T
4	4	3	1	1	0	1	1	-1	7	4/7	T
5	3	1	3	0	1	0	0	1	4	1/4	T
6	1	0	-	-	-	-	1	0	1	-	-